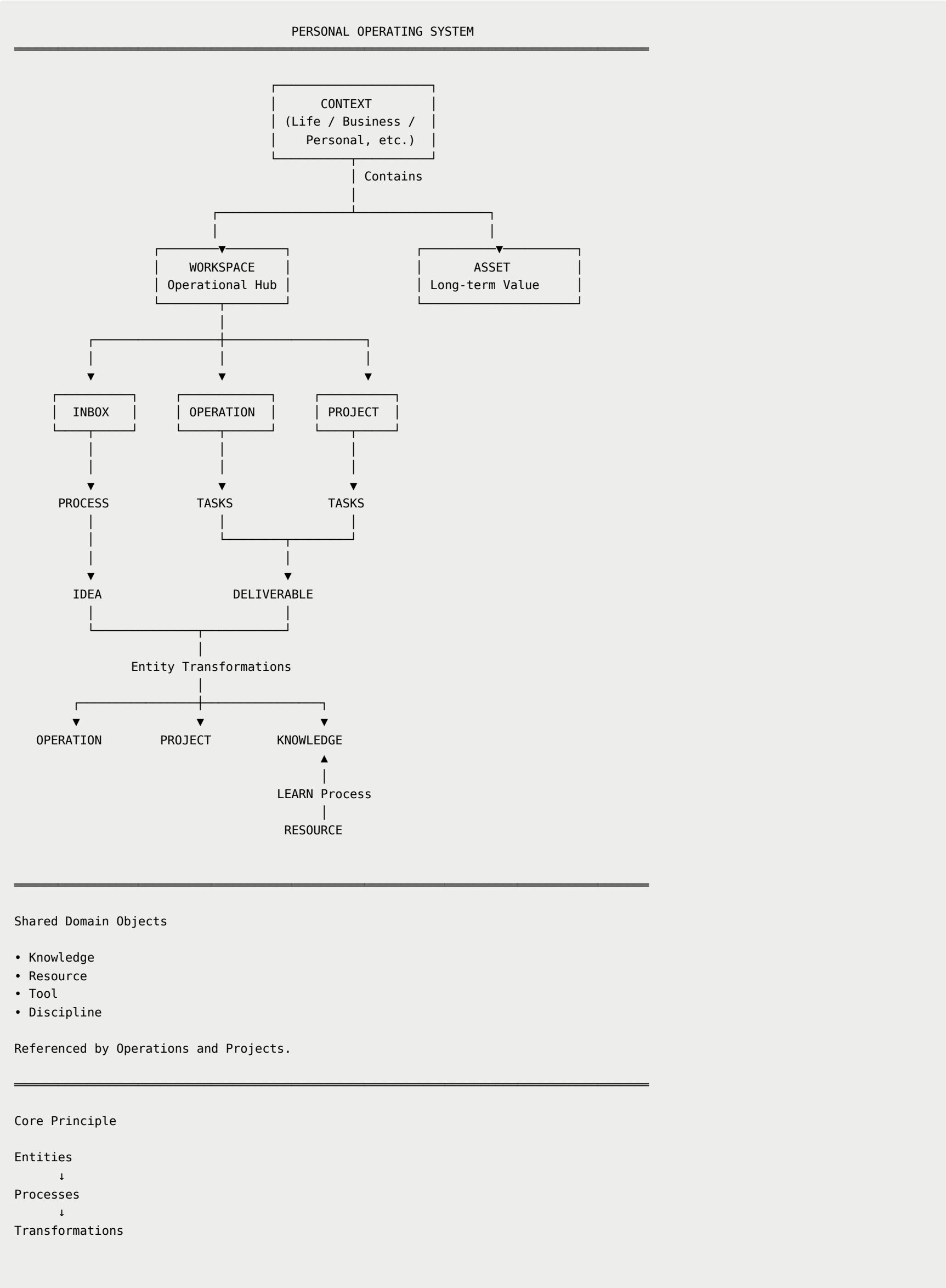




04. Domain Model

Domain Model — Conceptual Diagram v0.1



Resumen — Domain Model (v0.1)

Objetivo

El **Domain Model** define cómo se estructura y comporta el dominio del Personal Operating System. A diferencia del Ubiquitous Language, que establece el vocabulario, el Domain Model describe las relaciones, responsabilidades, restricciones y evolución de las entidades.

Su propósito es servir como base para la arquitectura, el flujo de información, el modelo de datos y cualquier implementación futura.

Objetivos específicos

- Identificar las entidades centrales del dominio.
- Establecer relaciones y ownership.
- Definir responsabilidades.
- Formalizar reglas invariantes.
- Modelar la evolución de las entidades.
- Proporcionar un modelo independiente de cualquier herramienta.

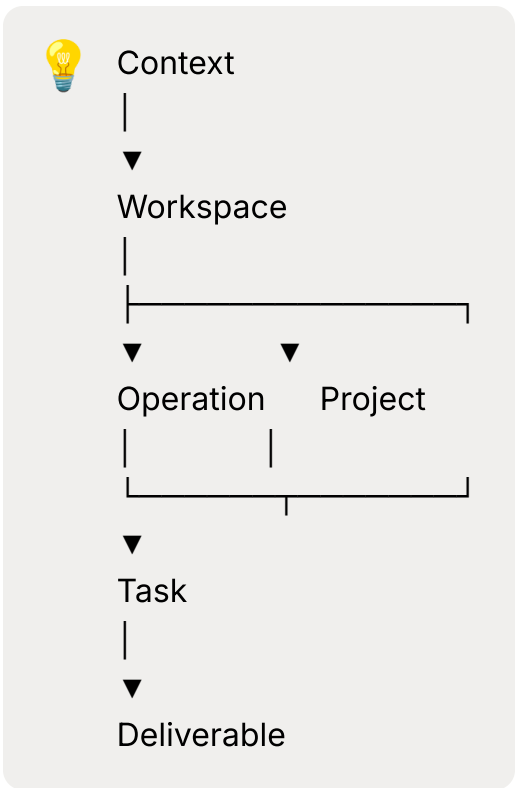
4.1 Aggregate Roots

Se identificaron como candidatos principales:

Entidad	Rol
Context	Delimita un dominio de responsabilidad.
Workspace	Centro operativo de un Context.
Operation	Representa una capacidad permanente.
Project	Representa una iniciativa temporal.

Se consideró que **Knowledge** y **Asset** podrían evolucionar hacia Aggregate Roots en futuras versiones, dependiendo del nivel de autonomía que adquieran.

Primera propuesta de relaciones



Observación arquitectónica

Aquí aparece una decisión que considero estratégica: distinguir entre entidades **contenedoras** y entidades **compartidas**.

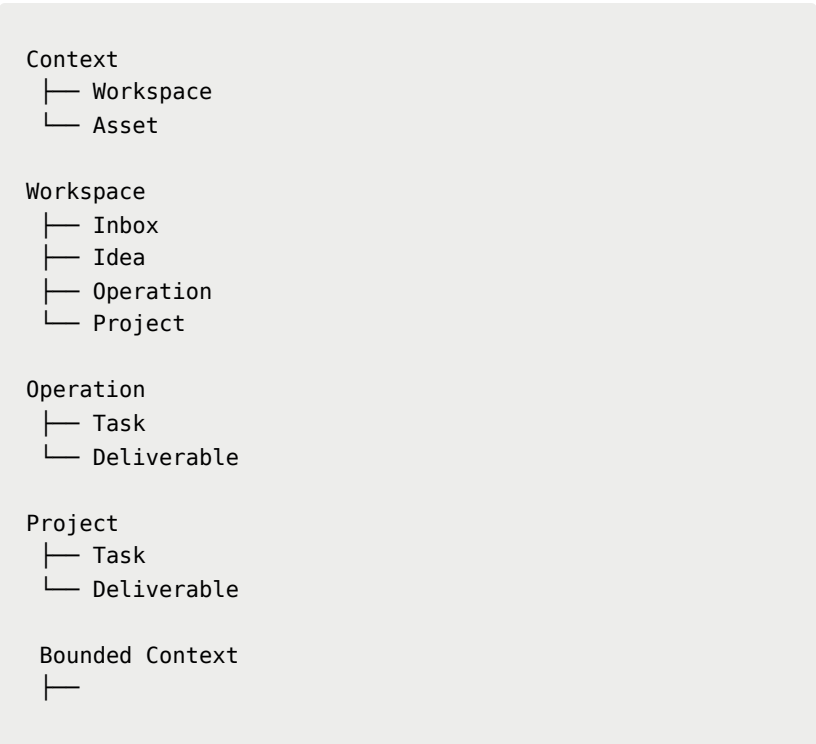
- **Context, Workspace, Operation y Project** estructuran el trabajo y poseen otras entidades.
- **Knowledge, Resource, Asset, Tool y Discipline** funcionan como recursos compartidos que pueden ser utilizados por múltiples operaciones y proyectos sin cambiar de propietario.

Esta separación reduce el acoplamiento y facilita que el POS evolucione hacia una arquitectura modular, donde los activos de conocimiento y los medios de trabajo no quedan "encerrados" dentro de un único flujo operativo.

4.2 Ownership

El ownership define qué entidad es responsable del ciclo de vida de otra.

Ownership principal

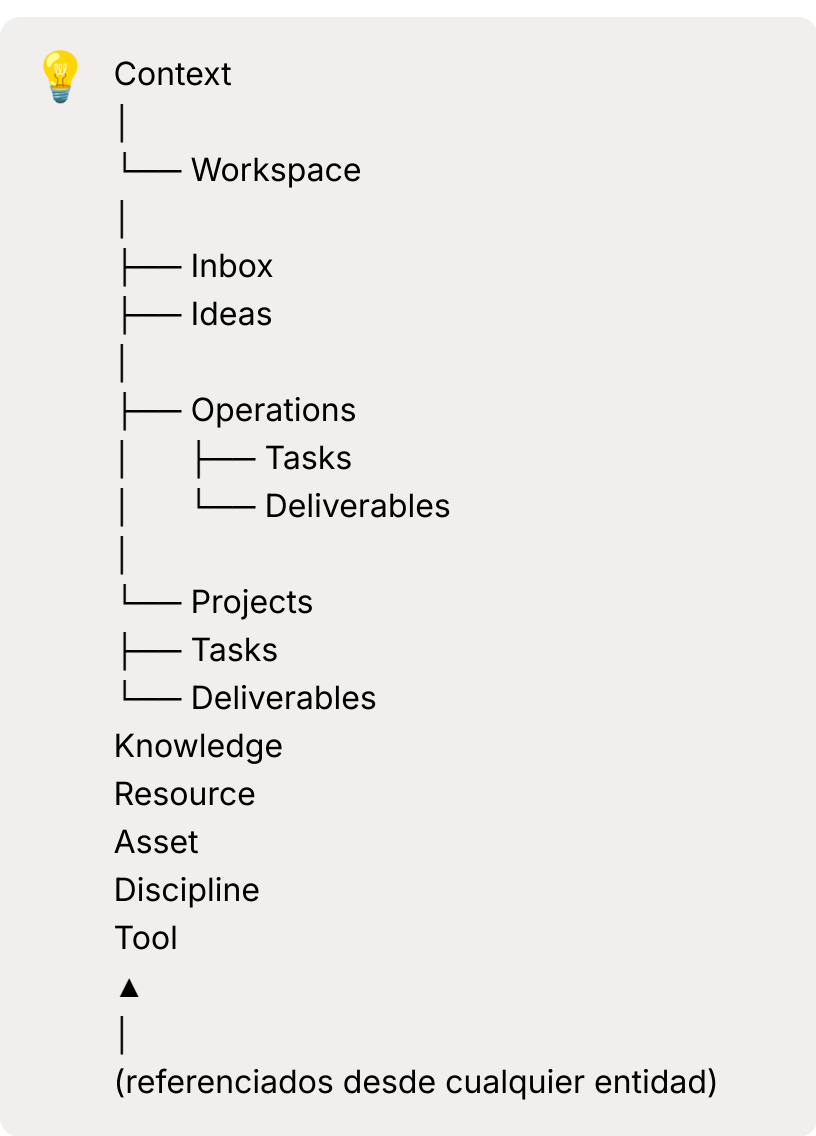


Entidades compartidas

Las siguientes entidades no pertenecen exclusivamente a otra entidad y pueden ser referenciadas desde múltiples partes del sistema:

- Knowledge
- Resource
- Tool
- Discipline

Modelo resultante



Observación arquitectónica

Durante este ejercicio aparece un patrón muy consistente: el dominio parece dividirse en **dos capas**.

1. **Capa operativa:** organiza y ejecuta el trabajo (Context, Workspace, Operation, Project, Task, Deliverable, Inbox, Idea).
2. **Capa de conocimiento:** provee capacidades reutilizables (Knowledge, Resource, Asset, Discipline, Tool).

Las entidades de la primera capa **poseen** el trabajo; las de la segunda **lo enriquecen** mediante referencias. Esta separación reduce el acoplamiento y facilita que el conocimiento evolucione independientemente de las operaciones que lo consumen.

4.3 Relaciones

Se definieron tres tipos principales.

Tipo	Significado
Contains	Forma parte de la estructura (ownership).
Uses	Utiliza otra entidad sin poseerla.

Tipo	Significado
Produces	Una entidad genera o transforma otra.

Relaciones estructurales (Contains)

Origen → Relación → Destino

Context → Contains → Workspace

Context → Contains → Asset

Workspace → Contains → Operation

Workspace → Contains → Project

Workspace → Contains → Inbox

Workspace → Contains → Idea

Operation → Contains → Task

Project → Contains → Task

Operation → Produces → Deliverable

Project → Produces → Deliverable

Relaciones funcionales (Uses)

Entidad → Usa

Operation → Knowledge, Resource, Asset, Tool, Discipline

Project → Knowledge, Resource, Asset, Tool, Discipline

Task → Resource, Tool

Learn (proceso) → Resource, Deliverable

Build (proceso) → Knowledge, Asset

Relaciones de transformación (Produces)

Capture → Inbox Item

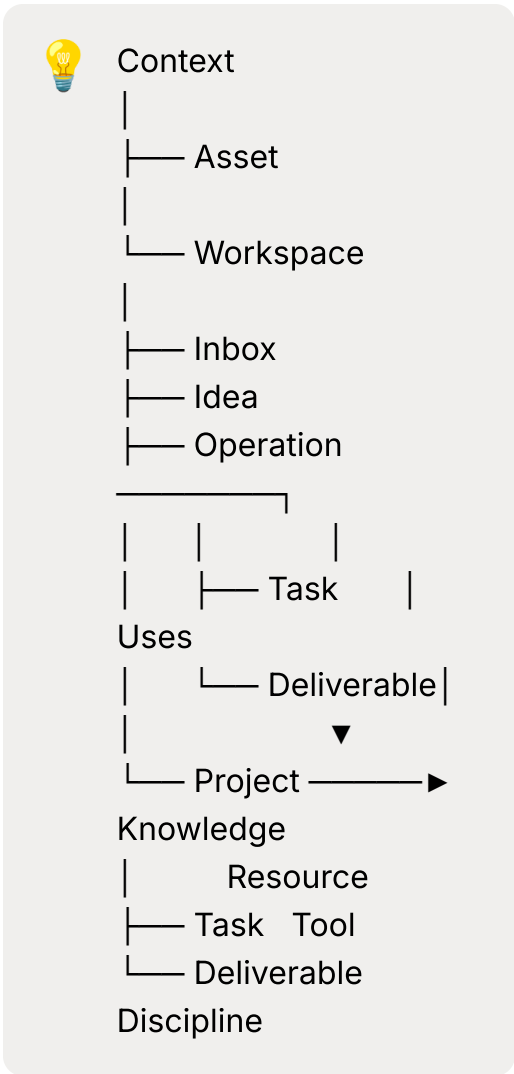
Process → Idea, Task, Resource, Knowledge (según el caso)

Learn → Knowledge

Build → Asset o Deliverable

Execute → Deliverable

Review → Nuevas Ideas, cambios o decisiones



Observación arquitectónica

Mientras definíamos estas relaciones, apareció una idea que puede convertirse en un principio del POS:

Las entidades operativas generan valor; las entidades compartidas amplifican ese valor.

En otras palabras:

- Una **Operation** crea resultados.
- Un **Project** crea resultados.
- Pero esos resultados son mejores porque utilizan **Knowledge, Assets, Tools y Resources**.

Eso sugiere que el crecimiento del sistema no dependerá de crear más operaciones, sino de enriquecer continuamente la base compartida de conocimiento y activos reutilizables. Es un comportamiento muy parecido al de un sistema operativo: las aplicaciones realizan el trabajo, mientras que el sistema proporciona servicios comunes que potencian a todas ellas.

04.x Entity Evolution Model

Desde → *Proceso* → Hacia

Inbox → *Process* → Idea

Inbox → *Process* → Task

Inbox → *Process* → Resource

Resource → *Learn* → Knowledge

Idea → *Process* → Project

Idea → *Process* → Operation

Idea → *Process* → Task

Project → *Build* → Deliverable

Deliverable → *Build* / Review → Asset

Project → *Learn* → Knowledge

Project → *Standardize* → Operation

Operation → *Execute* → Deliverable

Esta matriz sería el puente perfecto entre el **Domain Model** y el futuro **Information Flow**.



Observación arquitectónica

Creo que acabamos de encontrar uno de los principios más profundos del POS:

El sistema no está diseñado para almacenar entidades, sino para facilitar su evolución.

Las entidades son solo estados intermedios dentro de un flujo continuo de transformación. Un artículo puede convertirse en conocimiento; una idea en un proyecto; un entregable en un activo; un proyecto exitoso en una operación permanente. Si adoptamos esta perspectiva, el POS deja de ser un sistema de organización y pasa a ser un **sistema de evolución**, donde el valor no reside en lo que se guarda, sino en cómo madura con el tiempo.

4.5 Domain Invariants

Una **invariante** no es una recomendación ni una buena práctica.

Es una **regla del dominio** que, si se rompe, el sistema deja de representar correctamente la realidad.

Estructura

ID	Invariante	Justificación
INV-001	Todo Workspace pertenece a un único Context .	Mantiene la separación de dominios.
INV-002	Una Operation pertenece a un único Workspace .	Define su ámbito operativo.
INV-003	Un Project pertenece a un único Workspace .	Evita proyectos sin contexto.
INV-004	Un Task debe pertenecer a una Operation o a un Project .	No existen tareas huérfanas.
INV-005	Todo Deliverable debe originarse en una Operation o un Project .	Todo resultado tiene un origen trazable.

Conocimiento

ID	Invariante	Justificación
INV-009	El Knowledge solo puede originarse mediante un	Preserva la diferencia entre información y conocimiento.

Captura y organización

ID	Invariante	Justificación
INV-006	Todo elemento externo ingresa primero por el Inbox .	Unifica el punto de entrada del sistema.
INV-007	Ningún elemento permanece indefinidamente en el Inbox .	El Inbox es transitorio.
INV-008	Todo elemento capturado debe ser procesado o descartado.	Evita acumulación sin propósito.

Trabajo

ID	Invariante	Justificación
INV-012	Una Operation representa trabajo recurrente.	Define su naturaleza permanente.
INV-013	Un Project representa trabajo temporal.	Define su naturaleza finita.
INV-014	Una Task representa la unidad	Simplifica el modelo operativo.

ID	Invariante	Justificación
	proceso de aprendizaje (Learn).	
INV-010	Un Resource puede utilizarse múltiples veces sin perder su identidad.	Es un insumo reutilizable.
INV-011	Un Asset pertenece al Context y puede ser reutilizado por múltiples Workspaces.	Representa valor acumulado del contexto.

Transformación

ID	Invariante	Justificación
INV-015	Toda transformación debe preservar la trazabilidad entre origen y resultado.	Permite reconstruir la evolución del sistema.
INV-016	Una entidad puede transformarse en otra sin perder su historial.	Mantiene la continuidad del conocimiento.
INV-017	Ninguna transformación elimina información de forma implícita.	Evita pérdida accidental de contexto.

ID	Invariante	Justificación
	mínima de ejecución.	

Gobernanza

ID	Invariante	Justificación
INV-018	Ninguna Tool define el comportamiento del dominio.	Garantiza independencia tecnológica.
INV-019	Las reglas del dominio prevalecen sobre cualquier implementación.	La arquitectura es la fuente de verdad.
INV-020	Toda entidad debe tener un propósito dentro del sistema.	Evita elementos redundantes o sin función.



Observaciones arquitectónicas

Hay dos invariantes que destacaría especialmente porque reflejan la filosofía del proyecto:

INV-009

El conocimiento no se captura; se construye.

Esta regla diferencia claramente **Resource** de **Knowledge** y evita que cualquier información almacenada se considere conocimiento por defecto.

INV-015

Toda transformación debe ser trazable.

Este principio conecta con la filosofía de **Specs First, Docs as Code** y con el enfoque de un sistema orientado a evolución. Permite responder preguntas como:

- ¿De qué idea nació esta operación?
- ¿Qué recurso originó este conocimiento?
- ¿Qué proyecto generó este asset?

Observación arquitectónica

Mientras definíamos las invariantes apareció una idea que podría convertirse en un principio transversal del POS:

Nada aparece ni desaparece sin explicación.

Toda entidad tiene un origen identificable, un propósito dentro del sistema y un historial de transformaciones. Esto no solo mejora la trazabilidad; convierte al POS en un sistema donde el conocimiento y el trabajo son acumulativos. El valor no se pierde, se transforma.

4.6 Lifecycle

Creo que aquí conviene hacer una distinción importante.

Un **State** es un estado posible de una entidad.

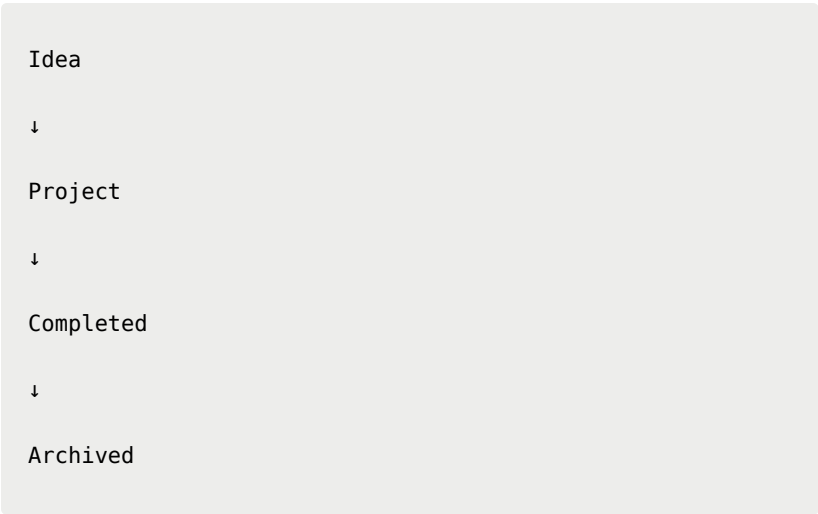
Un **Lifecycle** es el recorrido válido que esa entidad puede realizar entre esos estados y transformaciones.

Es decir:

- **States** = vocabulario.
- **Lifecycle** = comportamiento.

Entidad	Ciclo de vida
Idea	Captured → Processed → Planned → (Project Operation Task Knowledge Discarded)
Inbox Item	Captured → Processed → (Idea Task Resource Project Operation Discarded)
Task	Planned → In Progress → (On Hold Completed) → Archived
Project	Planned → In Progress → Completed → Archived
Operation	Planned → Active (<i>In Progress</i>) → Review → Improved (<i>vuelve a Active</i>) → Archived (<i>opcional</i>)
Deliverable	In Progress → Completed → Delivered → (Archived Asset)
Resource	Captured → Processed → Used → (Knowledge Archived)
Knowledge	Created → Applied → Improved → Archived (<i>opcional</i>)
Asset	Created → Used → Improved → Reused (<i>cíclico</i>) → Archived (<i>opcional</i>)

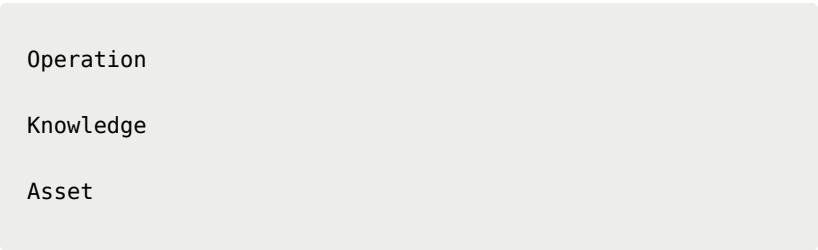
Ciclos finitos



Incluyen:

- Idea
- Task
- Project
- Deliverable

Ciclos evolutivos



No tienen un final esperado.

Su propósito es mejorar continuamente.

Incluyen:

- Operation
- Knowledge
- Asset

6. Una posible clasificación

Podríamos incluso documentar las entidades según su naturaleza:

Tipo	Entidades
Transient	Inbox Item, Idea
Finite	Task, Project, Deliverable
Persistent	Operation, Knowledge, Asset
Structural	Context, Workspace
Reference	Resource, Discipline, Tool

Esta clasificación complementa las categorías del Ubiquitous Language al describir el comportamiento temporal de las entidades.



Observación arquitectónica

Hay una simetría muy interesante entre el propósito del POS y los ciclos de vida que estamos definiendo:

- Las entidades **transitorias** capturan oportunidades.
- Las entidades **finitas** producen resultados.
- Las entidades **persistentes** acumulan capacidades y valor.
- Las entidades **estructurales** proporcionan estabilidad al sistema.

Esto refuerza la idea de que el POS no está diseñado para gestionar tareas o proyectos aislados, sino para **convertir trabajo efímero en capacidades permanentes**. Esa podría convertirse en una de las tesis centrales del sistema.

Principios arquitectónicos descubiertos

Durante el modelado del dominio surgieron varios principios fundamentales:



El dominio es una red de transformaciones

Las entidades no son estáticas; evolucionan y generan nuevas entidades.



Las operaciones generan valor

Los proyectos producen resultados específicos.
Las operaciones construyen capacidades permanentes.



El conocimiento se construye

No toda información es conocimiento.
El conocimiento surge cuando un recurso es comprendido, validado y reutilizado.



Los activos acumulan valor

Un Deliverable satisface una necesidad inmediata.
Un Asset incrementa el valor del sistema a largo plazo mediante la reutilización.



El POS organiza evolución

Más que almacenar información o tareas, el sistema facilita que las entidades evolucionen a través de procesos y transformaciones.

Resultado del Domain Model

Al finalizar esta fase, el POS dispone de un modelo conceptual compuesto por:

- Aggregate Roots identificados.
- Ownership definido.

- Relaciones entre entidades.
- Modelo de Transformaciones.
- Domain Invariants.
- Lifecycle de las entidades.
- Primer diagrama conceptual.

Este modelo constituye la base para las siguientes fases:

- **05. System Architecture**, donde se definirán los motores, componentes y límites del sistema.
 - **06. Information Flow**, donde se modelará el recorrido de la información y las entidades a través del POS.
 - **07. Conceptual Data Model**, donde el dominio se traducirá en un modelo de datos independiente de cualquier tecnología.
-

Observación final del arquitecto

Creo que durante esta fase apareció la idea más importante de todo el proyecto hasta ahora:

■ **El POS no modela objetos; modela la evolución del trabajo y del conocimiento.**

Ese cambio de perspectiva lo diferencia de metodologías como GTD, PARA, Notion o gestores de tareas tradicionales. En ellos, las entidades suelen ser el centro del sistema. En el POS, las entidades son simplemente vehículos dentro de un proceso continuo de transformación que convierte información en conocimiento, trabajo en capacidades y resultados en activos reutilizables. Si este principio se mantiene en las siguientes fases, probablemente se convierta en la esencia arquitectónica del sistema.